Introduction

If you have not done so already, please be sure to carefully read:

- overview
- description of the project structure

Phase 0 of the term project is a preparation phase. While all tasks below are due by the Phase 0 deadline, your submissions will only be graded with Phase 1.

Note that this first phase of the project does **not** require any programming yet. Instead, we want to encourage you to put as much thinking into your project idea as possible before you write the first line of code -- in practice, we are often tempted to immediately start writing code and this turns out to be a mistake.

Academic Honesty

Your project must be your own work. Phase 1 will include a more detailed, primarily code-oriented academic conduct policy, but the best start is to choose your own, personal topic in this phase. All content for your project must be content that you have written yourself since the start of this term.

Main Tasks

Start by cloning the repository project_xxx from GitHub where xxx is your CS ID. Then please be sure to read **all the way to the end of this page** before you do **any** work on your project!

Task 1

Your first task is decide what you'd like to create for your term project. We recommend that you give this some thought as you'll be working on the project that you select for the rest of the term. This may seem daunting but we'll be giving you guidelines along the way and your TA will be there to offer guidance. There are some broad criteria which you must adhere to in the design of your personal project:

- all code will be written in Java
- you will design a Java desktop application (no Android applications, for example)
- ultimately, your application will have a graphical user interface but you will start by designing a console based application (in phase 1)
- you will need to be able to save and restore the entire state of your application (so that the user could save, close the program, reopen the program, load, and continue from where they left off)
- to make the previous two points work, your application must have a model of the underlying data that is separate from the user interface; ideally, you could even save from your console-based application and then load into your graphical one or vice versa!
- your model must include a non-trivial class that you design to represent some important item in the data of your application and a different non-trivial class that you design that collects an arbitrary number of these items; many projects will have lots of classes that could fulfill these requirements

If you are stuck for ideas for your project, please try to discuss ideas with your TA first. If time goes by and you are still stuck, consider some examples:

- A pet tracking application for a shelter in a disaster location. It lets you enter information about pets brought to the shelter. Possible features include different species or breeds of pets; listing all pets confirmed to be available for adoption; showing a roster of pet pictures or icons. (X could be a pet, a species, a breed, etc. Y could be a list of pets or species, a breed classification category (like "Toy Dogs"), a client's preference list over pets, etc.)
- A system for recording games in your board gaming group. Enter results for each game, get statistics

- A system for recording games in your board gaming group. Enter results for each game, get statistics on how frequently games are played or win rates, group games by number of players involved. Possible features include a chart visualizing the history of your gaming group that makes it easy to see who played and won in each game. (X could be a board game, a play of a board game, a move in a game, a player, etc. Y could be someone's collection of games or wishlist they want to buy, a person's record of games they've played, a game made up of its moves, a gaming group of players, etc.)

The above should serve as examples that you could use as a last resort. But try to come up with your own idea for a term project, so that you have something unique to talk about at a future job interview. As this is a personal project, once the course is over, you'll be able to publish it on GitHub.com, so that others can view your work. For that reason, we strongly encourage you to think carefully about the design of your code - remember it's written for people to read (that includes a future employer!), as well as to be run on a computer.

Task 2

Use the Project view in IntelliJ to browse your project and note that it has a file named README.md. Your first task is to write one or two paragraphs at the start of this file that describe the project that you propose to design this term. Your proposal should provide answers to the following questions:

- What will the application do?
- Who will use it?
- Why is this project of interest to you?

The .md extension on the file name tells us that this is a *markdown* file. We're not expecting you to become markdown experts but you should at least learn how to:

- add a title and subtitle
- apply bold or italic fonts
- insert a bulleted list

You must make use of at least two of these three elements in your proposal. Note that the README file that we provide to you includes examples of each of these elements. A guide to these and other elements of basic markdown syntax can be found at https://www.markdownguide.org/basic-syntax/ - be sure to focus on the markdown elements and the corresponding rendered output but ignore the HTML. The use of markdown is prevalent and it is therefore well worth learning at least the basics. The course web page, for example, is almost exclusively written in markdown. Markdown is also used to format the feedback you receive from AutoTest. IntelliJ has a markdown renderer built in, so you will be able to preview your README page.

Note that the README page is displayed by default on your repository's homepage on GitHub - this is the first thing someone will see when they look at your repository, so it's worth putting some effort into making it look good.

When you are done, commit and push your code to GitHub. Remember to use a commit comment that accurately describes the changes that you have made in the current commit.

Task 3

During each phase of the term project, you will write *user stories* that describe how someone can use your application to produce a specific outcome. You will then design the code necessary to make that user story a reality in your application. Here are some examples of user stories:

For example, in the context of a pet tracking application:

- As a user, I want to be able to add a pet to my shelter
- As a user, I want to be able to view the list of pets in my shelter
- As a user, I want to be able to mark a pet as avaialable for adoption
- As a user, I want to be able to remove a pet from my shelter
- As a user, I want to be able to see the number of pets in my shelter and the number of pets that

have been adopted

For example, in the context of a board gaming group application:

- As a user, I want to be able to add a game to my gaming group
- As a user, I want to be able to view the list of games in my gaming group
- As a user, I want to be able to mark a game as played in my gaming group
- As a user, I want to be able to remove a game from my gaming group
- As a user, I want to be able to see statistics on how frequently games are played, including how frequently games are being played

For example, in the context of a to-do application:

- As a user, I want to be able to add a task to my to-do list
- As a user, I want to be able to view the list of tasks on my to-do list
- As a user, I want to be able to mark a task as complete on my to-do list
- As a user, I want to be able to delete a task from my to-do-list
- As a user, I want to be able to see the number of incomplete and number of completed tasks on my to-do list

In the context of a recipe manager application:

- As a user, I want to be able to add a recipe to my collection
- As a user, I want to be able to view a list of the titles of the recipes in my collection
- As a user, I want to be able to select a recipe in my collection and view the recipe in detail
- As a user, I want to be able to select a recipe in my collection and rate it on a scale of one to five stars

In the context of a fantasy sports team application:

- As a user, I want to be able to create a new team and add it to a list of teams
- As a user, I want to be able to select a team and add a new player to the team
- As a user, I want to be able to select a team and view a list of the players on that team
- As a user, I want to be able to select a player on a team and add a new statistic for that player, for example, the number of goals scored in a game
- As a user, I want to be able to select a team and list all the players on that team

Note that in each of these sample applications, there is at least one user story that involves the user adding an X to a Y (e.g. adding an item to a to-do list), where X and Y are classes that you will have to design yourself. The user must be able to do this an arbitrary number of times. By *arbitrary*, we mean that it will not be sufficient to always add a fixed number of Xs to a Y. In the context of a to-do list application, for example, X will be `Task` (a class which represents a single to-do item) and Y will be `ToDoList` (a class that stores multiple `Task` objects). In the context of a game, X might be `Treasure` and Y could be `Cache` (a class that stores

Previous Next

© All Rights Reserved

edX Logo